

**Министерство образования Российской Федерации**  
**Пензенский государственный университет**  
**Кафедра «Вычислительная техника»**

**Отчёт**

по лабораторной работе №9

по курсу «Л и О А в ИЗ»

на тему «Поиск расстояний в графе»

Выполнили студенты группы 24ВВВ4:

Кошелев Р.Д.

Кондратьев С.В.

Приняли к.т.н., доцент:

Юрова О.В.

к.э.н., доцент:

Акифьев И.В.

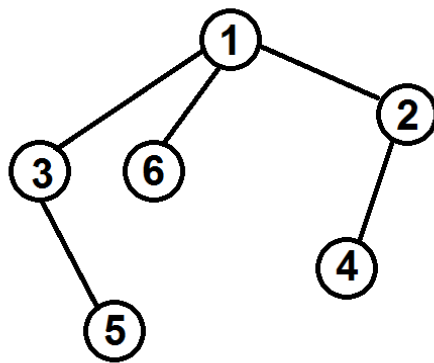
Пенза 2025

**Цель работы:** изучение представления графа с помощью матрицы смежности, реализация алгоритмов поиска расстояний от заданной вершины до всех остальных вершин на основе обхода в ширину и обхода в глубину, формирование вектора расстояний DIST и оценка времени работы этих алгоритмов на графах различного размера.

### **Общие сведения.**

Поиск расстояний – довольно распространенная задача анализа графов.

Для поиска расстояний можно использовать процедуры обхода графа. Для этого при каждом переходе в новую вершину необходимо запоминать, сколько шагов до нее мы сделали. При этом вектор, который хранил информацию о посещении вершин становится вектором расстояний. Довольно просто модернизировать для поиска расстояний в графе алгоритм обхода в ширину, т.к. этот алгоритм проходит вершины по уровням удаленности, то для не ориентированного графа для вершин каждого следующего уровня глубины расстояние от исходной вершины увеличивается на 1. Удалённость в данном случае понимается как количество ребер, по которым необходимо прейти до достижения вершины.



$$G = \begin{Bmatrix} 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{Bmatrix}$$

Рисунок 1 – Граф

Таким образом, можно предложить следующую реализацию алгоритма обхода в ширину.

Вход:  $G$  – матрица смежности графа,  $v$  – исходная вершина.

Выход:  $DIST$  – вектор расстояний до всех вершин от исходной.

Алгоритм ПОШ

- 1.1. для всех  $i$  положим  $DIST[i] = -1$  пометим как "не посещенную";
- 1.2. ВЫПОЛНЯТЬ BFSD ( $v$ ).
- 1.3 для всех  $i$  вывести  $DIST[i]$  на экран;

Алгоритм BFSD( $v$ ):

- 2.1. Создать пустую очередь  $Q = \{\}$ ;
- 2.2. Поместить  $v$  в очередь  $Q.push(v)$ ;
- 2.3. Обновить вектор расстояний  $DIST[x] = 0$ ;
- 2.4. ПОКА  $Q \neq \emptyset$  очередь не пуста ВЫПОЛНЯТЬ
- 2.5.  $v = Q.front()$  установить текущую вершину;
- 2.6. Удалить первый элемент из очереди  $Q.pop()$ ;
- 2.7. вывести на экран  $v$ ;
- 2.8. ДЛЯ  $i = 1$  ДО  $size\_G$  ВЫПОЛНЯТЬ
- 2.9. ЕСЛИ  $G(v,i) = 1$  И  $DIST[i] = -1$
- 2.10. ТО
- 2.11. Поместить  $i$  в очередь  $Q.push(i)$ ;
- 2.12. Обновить вектор расстояний  $DIST[i] = DIST[v] + 1$ ;

Реализация состоит из подготовительной части, в которой все вершины помечаются как не посещенные (п.1.1). В отличие от алгоритма BFS не посещенные вершины помечаем -1, т.к. значение 0 и 1 могут быть расстояниями. Расстояние 0 – от исходной вершины до самой себя.

В самой процедуре как и в алгоритме BFS сначала создается пустая очередь (п. 2.1), в которую помещается исходная вершина, из которой начат

обход (п.2.2). Расстояние до этой вершины (п.2.3) устанавливается равным 0 (расстояние до самой себя).

Далее итерационно, пока очередь не опустеет, из нее извлекается первый элемент, который становится текущей вершиной (п. 2.5, 2.6). Затем в цикле просматривается  $v$ -я строка матрицы смежности графа  $G(v,i)$ . Как только алгоритм встречает смежную с  $v$  не посещенную вершину (п.2.9), эта вершина помещается в очередь (п.2.11) и для нее обновляется вектор расстояния (п.2.12). Расстояние до новой  $i$ -й вершины вычисляется как расстояние до текущей  $v$ -й вершины плюс 1 (так как ребра нашего графа не взвешенные).

После просмотра строки матрицы смежности алгоритм делает следующую итерацию цикла 2.4 или заканчивает работу, если очередь пуста.

Таким образом, если вершина помещается в очередь при просмотре строки матрицы смежности на 1-й итерации, то они находятся на 1 уровне удаленности и расстояние до этих вершин будет равным 1.

$DIST[i] = DIST[v] + 1$ , где  $DIST[v] = 0$  – расстояние от исходной вершины до самой себя.

Далее, начинают просматриваться вершины первого уровня и соответствующие им строки матрицы смежности. При добавлении смежных с вершинами первого уровня вершин, расстояния до них будут равны 2.

$DIST[i] = DIST[v] + 1$ , где  $DIST[v] = 1$  – расстояние от исходной вершины до вершин 1 уровня.

После того, как все вершины первого уровня будут просмотрены и извлечены из очереди, начнется просмотр вершин 2 уровня и соответствующих им строк матрицы смежности. При добавлении смежных с вершинами второго уровня вершин, расстояния до них будут равны 3.

$DIST[i] = DIST[v] + 1$ , где  $DIST[v] = 2$  – расстояние от исходной вершины до вершин 2 уровня.

И так далее, алгоритм проходит вершины по уровням, пока очередь не опустеет.

## **Задание 1**

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа  $G$ . Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс `queue` из стандартной библиотеки C++.
3. \* Реализуйте процедуру поиска расстояний для графа, представленного списками смежности.

### **Задание 2\***

1. Реализуйте процедуру поиска расстояний на основе обхода в глубину.
2. Реализуйте процедуру поиска расстояний на основе обхода в глубину для графа, представленного списками смежности.
3. Оцените время работы реализаций алгоритмов поиска расстояний на основе обхода в глубину и обхода в ширину для графов разных порядков.

## Результат работы программы

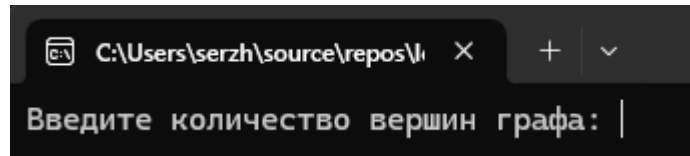


Рисунок 1 – Ввод количества вершин для неориентированного графа.

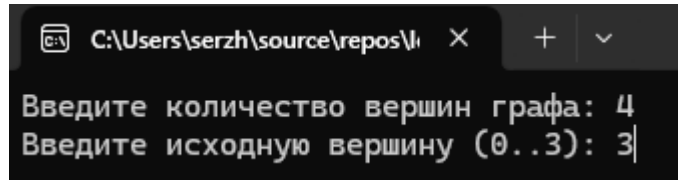


Рисунок 2 – Ввод исходной вершины для неориентированного графа.

```
Матрица смежности:
1 1 0 0
1 1 0 1
0 0 1 1
0 1 1 1

BFS по матрице, порядок обхода: 3 1 2 0

DIST BFS (матрица):
DIST[0]=2
DIST[1]=1
DIST[2]=1
DIST[3]=0
Время BFS (матрица): 1,000 мс
```

Рисунок 3 – Вывод матрицы смежности, обход графа в ширину по матрице(BSF), вектора расстояний для матрицы для алгоритма обхода в ширину(DIST BFS) и времени обхода матрицы в ширину.

```
DIST DFS (матрица):
DIST[0]=2
DIST[1]=1
DIST[2]=1
DIST[3]=0
Время DFS (матрица): 0,000 мс

Списки смежности графа:
0:
1: 3 2
2: 3 2 2 1
3: 3 3 2 1

BFS по спискам смежности, порядок обхода: 3 2 1
```

Рисунок 4 – Вывод вектора расстояний для матрицы для алгоритма обхода в глубину(DIST DFS), времени обхода матрицы в глубину, списка смежности графа и порядка обхода по спискам в ширину.

```
DIST BFS (списки):  
DIST[0]=-1  
DIST[1]=1  
DIST[2]=1  
DIST[3]=0  
Время BFS (списки): 1,000 мс  
  
DIST DFS (списки):  
DIST[0]=-1  
DIST[1]=2  
DIST[2]=1  
DIST[3]=0  
Время DFS (списки): 0,000 мс
```

Рисунок 4 – Вывод вектора расстояний для списка смежности для алгоритма обхода в ширину(DIST BFS), времени обхода матрицы в ширину, вектора расстояний для списка смежности для алгоритма обхода в глубину(DIST DFS), времени обхода матрицы в глубину.

**Вывод:** в ходе работы были успешно реализованы алгоритмы обхода в ширину и обхода в глубину для графа, представленного матрицей смежности, при этом BFS гарантирует минимальные расстояния до всех вершин, а DFS показывает расстояния вдоль выбранного пути, которые могут отличаться от минимальных. Замеры времени показали, что оба алгоритма работают эффективно для графов среднего размера, а работа позволила закрепить навыки работы с матрицей смежности, алгоритмами BFS и DFS и динамическим выделением памяти в C/C++.

## Приложение А

### Листинг

```
#define _CRT_SECURE_NO_WARNINGS

#include <locale.h>

#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#include <ctype.h>

#include <string.h>

#include <queue>

using namespace std;

/* Проверка, что введено положительное число */
int is_positive_integer(const char* str) {
    if (str == NULL || *str == '\0') return 0;
    for (int i = 0; str[i]; i++)
        if (!isdigit((unsigned char)str[i]))
            return 0;
    return atoi(str) > 0;
}

int** generateAdjMatrix(int n) {
    int** G = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++)
        G[i] = (int*)malloc(n * sizeof(int));

    for (int i = 0; i < n; i++) {
```



```

        for (int j = i; j < n; j++) {
            int edge = rand() % 2;
            G[i][j] = edge;
            G[j][i] = edge;
        }
    }
    return G;
}

```

/\* Печать матрицы \*/

```

void printMatrix(int** G, int n) {
    printf("\nМатрица смежности:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            printf("%d ", G[i][j]);
        printf("\n");
    }
}

```

/\* BFS по матрице \*/

```

void BFS_matrix(int** G, int n, int start, int* DIST) {
    for (int i = 0; i < n; i++) DIST[i] = -1;

    queue<int> Q;
    DIST[start] = 0;
    Q.push(start);
}

```

```

printf("\nBFS по матрице, порядок обхода: ");
while (!Q.empty()) {
    int v = Q.front(); Q.pop();
    printf("%d ", v);
    for (int i = 0; i < n; i++) {
        if (G[v][i] == 1 && DIST[i] == -1) {
            DIST[i] = DIST[v] + 1;
            Q.push(i);
        }
    }
}
printf("\n");
}

/* DFS по матрице */
void DFS_matrix(int** G, int n, int v, int* DIST, int dist) {
    DIST[v] = dist;
    for (int i = 0; i < n; i++) {
        if (G[v][i] == 1 && DIST[i] == -1) {
            DFS_matrix(G, n, i, DIST, dist + 1);
        }
    }
}

/* --- Структуры для списка смежности --- */
typedef struct Node {
    int vertex;

```

```

    struct Node* next;
} Node;

/* Построение списков смежности из матрицы смежности */
Node** buildAdjListFromMatrix(int** G, int n) {
    Node** adjList = (Node**)malloc(n * sizeof(Node*));
    for (int i = 0; i < n; i++) adjList[i] = NULL;

    for (int i = 0; i < n; i++) {
        for (int j = n - 1; j >= 0; j--) { // j с конца — для аккуратного порядка
            if (G[i][j] == 1) {
                Node* node = (Node*)malloc(sizeof(Node));
                node->vertex = j;
                node->next = adjList[i];
                adjList[i] = node;
            }
        }
    }
    return adjList;
}

/* Печать списков смежности */
void printGraph(Node** adjList, int n) {
    printf("\nСписки смежности графа:\n");
    for (int i = 0; i < n; i++) {
        printf("%d:", i);
        Node* curr = adjList[i];
    }
}

```

```

while (curr) {
    printf(" %d", curr->vertex);
    curr = curr->next;
}
printf("\n");
}
}

```

/\* BFS по спискам смежности \*/

```

void BFS_list(Node** adjList, int n, int start, int* DIST) {
    for (int i = 0; i < n; i++) DIST[i] = -1;

```

```

    queue<int> Q;

```

```

    DIST[start] = 0;

```

```

    Q.push(start);

```

```

    printf("\nBFS по спискам смежности, порядок обхода: ");

```

```

    while (!Q.empty()) {

```

```

        int v = Q.front(); Q.pop();

```

```

        printf("%d ", v);

```

```

        Node* curr = adjList[v];

```

```

        while (curr) {

```

```

            int u = curr->vertex;

```

```

            if (DIST[u] == -1) {

```

```

                DIST[u] = DIST[v] + 1;

```

```

                Q.push(u);
            }
        }
    }
}

```

```

    }

    curr = curr->next;

}

}

printf("\n");
}

```

/\* DFS по спискам смежности \*/

```

void DFS_list(Node** adjList, int v, int* DIST, int dist) {
    DIST[v] = dist;
    Node* curr = adjList[v];
    while (curr) {
        int u = curr->vertex;
        if (DIST[u] == -1) {
            DFS_list(adjList, u, DIST, dist + 1);
        }
        curr = curr->next;
    }
}

```

/\* Освобождение памяти \*/

```

void freeGraph(Node** adjList, int n) {
    for (int i = 0; i < n; i++) {
        Node* curr = adjList[i];
        while (curr) {
            Node* tmp = curr;
            curr = curr->next;

```

```

        free(tmp);
    }
}
free(adjList);
}

int main(void) {

    setlocale(LC_ALL, "rus");
    srand((unsigned)time(NULL));

    char input[100];
    int n;

    /* Ввод количества вершин */
    while (1) {
        printf("Введите количество вершин графа: ");
        if (!fgets(input, sizeof(input), stdin)) { fprintf(stderr, "Ошибка ввода\n");
return 1; }
        input[strcspn(input, "\n")] = '\0';
        if (is_positive_integer(input)) { n = atoi(input); break; }
        printf("Ошибка: нужно положительное число!\n");
    }

    /* Ввод стартовой вершины */
    int start;
    while (1) {
        printf("Введите исходную вершину (0..%d): ", n - 1);

```

```

        if (!fgets(input, sizeof(input), stdin)) { fprintf(stderr, "Ошибка ввода\n");
return 1; }

        input[strcspn(input, "\n")] = '\0';

        if (is_positive_integer(input) || strcmp(input, "0") == 0) {

            start = atoi(input);

            if (start >= 0 && start < n) break;

        }

        printf("Ошибка: некорректный номер вершины!\n");

    }

```

```

int* DIST = (int*)malloc(n * sizeof(int));

```

```

/* Матрица смежности */

```

```

int** G = generateAdjMatrix(n);

```

```

printMatrix(G, n);

```

```

/* BFS + время */

```

```

clock_t t1 = clock();

```

```

BFS_matrix(G, n, start, DIST);

```

```

clock_t t2 = clock();

```

```

double timeBFS_matrix = (double)(t2 - t1) / CLOCKS_PER_SEC * 1000;

```

```

printf("\nDIST BFS (матрица):\n");

```

```

for (int i = 0; i < n; i++) printf("DIST[%d]=%d\n", i, DIST[i]);

```

```

printf("Время BFS (матрица): %.3f мс\n", timeBFS_matrix);

```

```

/* DFS + время */

```

```

for (int i = 0; i < n; i++) DIST[i] = -1;

```

```

t1 = clock();
DFS_matrix(G, n, start, DIST, 0);
t2 = clock();
double timeDFS_matrix = (double)(t2 - t1) / CLOCKS_PER_SEC * 1000;

printf("\nDIST DFS (матрица):\n");
for (int i = 0; i < n; i++) printf("DIST[%d]=%d\n", i, DIST[i]);
printf("Время DFS (матрица): %.3f мс\n", timeDFS_matrix);

/* --- Списки смежности по матрице --- */
Node** adjList = buildAdjListFromMatrix(G, n);
printGraph(adjList, n);

/* BFS по спискам */
for (int i = 0; i < n; i++) DIST[i] = -1;
t1 = clock();
BFS_list(adjList, n, start, DIST);
t2 = clock();
double timeBFS_list = (double)(t2 - t1) / CLOCKS_PER_SEC * 1000;

printf("\nDIST BFS (списки):\n");
for (int i = 0; i < n; i++) printf("DIST[%d]=%d\n", i, DIST[i]);
printf("Время BFS (списки): %.3f мс\n", timeBFS_list);

/* DFS по спискам */
for (int i = 0; i < n; i++) DIST[i] = -1;
t1 = clock();

```



```
DFS_list(adjList, start, DIST, 0);

t2 = clock();

double timeDFS_list = (double)(t2 - t1) / CLOCKS_PER_SEC * 1000;

printf("\nDIST DFS (списки):\n");
for (int i = 0; i < n; i++) printf("DIST[%d]=%d\n", i, DIST[i]);
printf("Время DFS (списки): %.3f мс\n", timeDFS_list);

/* Освобождение памяти */
free(DIST);
freeGraph(adjList, n);
for (int i = 0; i < n; i++) free(G[i]);
free(G);

return 0;
}
```